

เอกสารนำเสนอโครงการ: ระบบจัดการสินค้าคงคลังแบบ CLI (v3.0)

1. ภาพรวมโครงการ

ระบบจัดการสินค้าคงคลังแบบ CLI สำหรับร้านค้าขนาดเล็ก ครอบคลุมงานรับสินค้าเข้า ตัดสต็อก แจ้งเตือนสินค้าใกล้หมด ระบบสมาชิกพร้อมส่วนลดอัตโนมัติ และรายงาน CSV พัฒนาต่อเนื่องจากโค้ดเดิมที่เป็นแบบ Monolithic ใช้ตัวแปลโทลบอล ☒ เก็บข้อมูล ในไฟล์ JSON ไม่มีการตรวจสอบข้อมูลนำเข้า โปรแกรมล่มเมื่อกรอกผิดประเภทและยอมรับ ค่าผิดพลาดได้

2. ทีมและบทบาท

ชื่อ-นามสกุล	รหัส	บทบาท	หน้าที่หลัก
ภาณุวัฒน์ คำคำ	6754210044-3	Project Manager / Developer	บริหารภาพรวม กำหนดกรอบเวลา พัฒนา Input Validation และการจัดเก็บข้อมูล
เอกพันธ์ ทศทิศรังสรรค์	67543210050-0	QA / Tester	ออกแบบ Test Cases และ Automated Unit Testing
ณฐภาพ สายหล้า	67543210054-2	Tech Lead / Architect	ออกแบบ Refactoring Architecture, Code Review, มาตรฐาน PEP 8

3. วัตถุประสงค์และขอบเขต

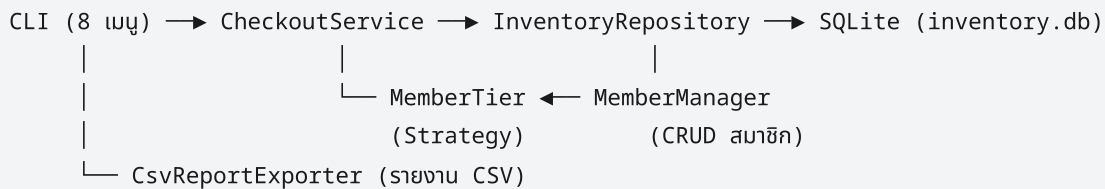
- เสถียรภาพ** — ไม่ล่มจากข้อมูลนำเข้าผิดประเภท กันค่าติดลบ ป้องกันข้อมูลสูญหาย ขณะบันทึก
- โครงสร้างสะอาด** — แยก Business Logic ออกจาก CLI ตาม OOP/PEP 8 เลิกใช้ global state
- วิวัฒนาการ** — ย้าย JSON → SQLite (Transactions), ระบบสมาชิก 4 tiers, Checkout คิดส่วนลดอัตโนมัติพร้อมใบเสร็จ
- คุณภาพ** — Unit/Integration Test ครอบคลุมตรรกะหลัก ส่วนลด และความปลอดภัย ฐานข้อมูล

4. เส้นทางวิวัฒนาการของระบบ

เวอร์ชัน	ฐานข้อมูล	สถาปัตยกรรม	ฟีเจอร์หลัก
v1.0	JSON (เขียนกับตรง)	Monolithic + global ☒	CLI 5 เมนู
v2.0	JSON (Atomic Save)	OOP 3 คลาส แยก Logic/CLI	Validation, กันติดลบ, Barcode, Reorder Point, CSV

เวอร์ชัน	ฐานข้อมูล	สถาปัตยกรรม	ฟีเจอร์หลัก
v3.0	SQLite (ACID)	Singleton + Repository + Strategy	Member 4 tiers, Checkout + ส่วนลด, Migration

5. สถาปัตยกรรม v3.0



- **Singleton** (`SQLiteDatabaseContext`) — จุดเชื่อมต่อฐานข้อมูลจุดเดียว ป้องกัน Database Locked
- **Repository** (`InventoryRepository`) — ช้อน SQL ไร้ชั้นเดียว ทุกคำสั่งใช้ parameterized query ป้องกัน SQL injection 100%
- **Strategy** (`MemberTier`) — Regular 0%, Silver 5%, Gold 10%, Platinum 15% เพิ่ม tier ใหม่ไม่ต้องแก้ if-else เติม
- ตาราง `products` (`product_id`, `name`, `quantity`, `price`, `category`, `barcode`, `reorder_point` พร้อม CHECK กันค่าติดลบ) และ `members` (`member_id`, `name`, `tier`, `discount_rate`)

6. ฟีเจอร์และการใช้งาน

เมนูหลัก 8 ข้อ: ดูสินค้า, เพิ่ม/แก้ไข (รวม barcode + reorder point), ตัดสต็อก, สรุปคลัง, ส่งออก CSV, จัดการสมาชิก, Checkout พร้อมส่วนลด, ออก

ตัวอย่างใบเสร็จ (สมาชิก Gold ซื้อสินค้า 500 บาท × 2):

Product : Gold Item
Qty x Price: 2 x 500.00
Subtotal : 1000.00 THB
Tier Gold (10%): -100.00 THB
TOTAL : 900.00 THB
Remaining stock: 8

- ไม่กรอกรหัสสมาชิก (หรือรหัสไม่มีในระบบ) = ราคาเต็มแบบ guest
- tier ที่ไม่รู้จักถูกปรับเป็น Regular โดยไม่ error
- สต็อกถึงจุดสั่งซื้อซ้ำมีค่าเตือนทั้งตอนตัดสต็อกและ checkout
- รายงาน CSV มีหัว `ProductID,ProductName,Barcode,Quantity,ReorderPoint,Price`
- ครั้งแรกที่รัน ระบบย้าย `data.json` เดิม (รวมคีย์เก่า `n/q/p/c`) เข้า SQLite อัตโนมัติพร้อมตรวจสอบว่าจำนวนและมูลค่าตรงกัน 100%

7. การเปลี่ยนแปลงสำคัญระหว่างทาง

รายการ	สาระ	ผล
CR-01	เพิ่ม Barcode + Reorder Point + แจ้งเตือน <code>quantity <= reorder_point</code>	ส่งใน v2.0
CR-02	ส่งออก CSV ดูกเงิน (ผ่าน CCB)	<code>CsvReportExporter</code> แยกคลาสตาม SRP
BUG-101	<code>KeyError: barcode</code> เมื่อเจอไฟล์ JSON เก่า	<code>from_dict</code> ใช้ <code>dict.get</code> + คำเริ่มต้น ну ข้อมูลเก่า

8. การทดสอบ

ชุดทดสอบ	ผล	ครอบคลุม
<code>test_app.py</code> (รอก, v3.0) + <code>web/backend/test_web_api.py</code>	20 passed	สรุปคลัง, ตัดสต็อก (สำเร็จ/ของไม่พอ/ไม่มีสินค้า), เพิ่ม-อัปเดต, Singleton, กัน injection, migration, Member CRUD 4 tiers, fallback, checkout (Gold/guest/alert), legacy JSON
Phase4/Sprint4/week-12/ <code>test_app.py</code> (regression v2.0)	25 passed	baseline + CR-01/CR-02 + BUG-101 + CLI + integration

- input อันตราย (`' OR '1'='1` , `DROP TABLE`) ถูกปฏิบัติเป็น string ธรรมดา ตารางไม่เสียหาย ยอดเงินเท่าเดิม
- ทดสอบ E2E: JSON ผสมคีย์เก่า/ใหม่ → migrate ตรง 100% → สมัคร Gold → checkout 1,000 → สุกี้ 900

9. คุณภาพโค้ด

- `flake8 app.py` ผ่านสะอาด (ไร้ E501) `bandit` ไม่พบประเด็น
- บันทึกแบบ atomic (ไฟล์ชั่วคราว + `os.replace`) ฝั่ง JSON เดิม และ commit/rollback ฝั่ง SQLite
- จับ exception แบบเจาะจง ไม่กลืน error

10. การบริหารโครงการ (1 sprint ต่อ 1 Phase)

Sprint	Phase	Issues	SP
1 (ID 84)	Initiation W1–4	SPM-6...10	15
2 (ID 82)	Design W5–7	SPM-11...14	13
3 (ID 85)	Execution W8–11	SPM-15...19	16
4 (ID 83)	Release W12 v2.0	SPM-20, 21	5
5 (ID 86)	Evolution v3.0 + Web	SPM-23...27, SPM-29...32	46

ทุก sprint ปิดแล้ว ทุก issue Done (sprint เก่า ID 45/48/81 เก็บเป็น ARCHIVED)

- จบฐานที่อนุมัติ 18,823 บาท ภาพรวมสุดท้ายตรงเวลา เกินงบ 575 บาท (3.7%) ซึ่งอยู่ในเงินสำรอง (ตัวเลขเพื่อการเรียน)

11. ความเสี่ยงและงานคงเหลือ

- ข้อมูล JSON เดิมแก้ด้วย fallback ไม่ใช่ migration เติมรูปแบบ (ฝั่ง v3.0 ใช้สคริปต์ย้ายพร้อมตรวจแทน)
- UAT รอบ formal sign-off และ Git tag รัลียังรอขึ้นตอนพิธีการ
- แนวทางต่อ: สมาชิก tier ใหม่ (เช่น Diamond) เติมได้ทันทีด้วย Strategy, ต่อยอด Web UI สำเร็จแล้ว (FastAPI + SPA) โดยไม่แตะชั้น Repository

12. สรุป

ระบบขยับจากสคริปต์ Monolithic ที่ลุ่มง่าย สู่สถาปัตยกรรมแบบชั้น (SQLite + Member + Checkout) ที่ทดสอบอัตโนมัติครอบคลุม 20 + 25 เคส ปลอดภัยจาก SQL injection ทนข้อมูลเก่า และพร้อมสาริตการทำงานจริงทุกเมนู

เอกสารอ้างอิง

- โค้ด: `app.py` , `test_app.py` (snapshot ใน `Phase4/` , `Phase5/`)
- รายงาน Sprint: `Phase1/Sprint1/sprint1.md` , `Phase2/Sprint2/sprint2.md` , `Phase3/Sprint3/sprint3.md` , `Phase4/Sprint4/sprint4.md` , `Phase5/Sprint5/sprint5.md`
- แบบออกแบบ: `Phase1/Sprint1/week-2/blueprint.md` , `Phase1/Sprint1/week-2/Member-Discout.md` , `Phase2/Sprint2/week-6/To_Be_Architecture.md`